

系统研发行动手册

从后端底座、Durable Agent Runtime，到微调/蒸馏、vLLM Serving、评测与可部署项目

01

后端与基础

02

Agent Runtime

03

可靠性与评测

04

模型训练与蒸馏

05

Serving 与部署

如何使用这本手册

先选节奏，再用一个主项目把所有能力串起来。

4 周求职冲刺

适合：已有后端/Agent 项目基础。

主线：Runtime → 数据与 Eval → LoRA/蒸馏 + vLLM → 集成演示。

产出：可恢复、可观测、可部署的主项目。

8 周完整闭环

适合：希望同时补 Agent Infra 与大模型工程。

主线：4 周 Runtime + 4 周模型原理、训练、微调、Serving。

产出：平台 + 小模型路由器 + 性能实验。

12 周基础优先

适合：Python 后端、网络/操作系统或分布式基础仍不稳。

主线：3 周后端 → 3 周单 Agent → 3 周 workflows 恢复 → 3 周生产化评测。

唯一主项目原则

Secure Durable Agent Platform

业务壳可选择 企业 Workflow / Coding / DevOps / Research / Data Agent；底层共享同一套任务状态、Worker、幂等、审批、Memory、Trace、Eval、模型路由和部署。

三个短实验只作为 labs/：Tiny Transformer、Tool-Routing Distillation、vLLM Gateway Benchmark。不要把简历堆成多个互不相关的半成品。

内容地图

从系统可靠性到模型训练，再回到可面试项目。

PART I

Durable Agent Infra

p.4

知识删减、异步任务、恢复、幂等、审批、可观测性、Kubernetes、vLLM Gateway。

PART II

Agent 与大模型扩展

p.16

Tool/Skills、安全、Eval 数据飞轮、Transformer、训练、LoRA/QLoRA、DPO、Serving。

PART III

练手项目与集成

p.25

一主三辅、工具路由蒸馏、模型路由、故障联动、仓库结构与完成标准。

PART IV

基础补齐与场景选择

p.32

能力地图、三档路线、8 类高价值 Agent 场景、项目选型与终版自检。

完整闭环

请求 → Task/Workflow → Worker/Tool/Sandbox → Context & Memory → Model Router → vLLM/强模型回退 → Trace/Eval → Badcase/Data → 再训练

一页结论

速成不是学完整个 AI Infra。最短路径是只攻一个可演示的垂直切面：把已有 Agent 改造成一个可恢复、可观测、可部署、可评测的执行平台。

最终项目	最短周期
Durable Agent Runtime：异步任务、状态恢复、工具幂等、人工审批、Trace、分层 Memory、K8s 弹性扩缩。	4 周完成 Agent Infra 主体；再用 1-2 周补 vLLM 推理网关与性能实验。
学习原则	面试卖点
只学项目当下需要的最小知识；每学一个概念，立即实现并制造故障验证。	不是“会调模型”，而是能解释 Agent 如何安全、可靠、可恢复、可观测地运行。

最终演示必须做到

- 创建一个 Agent 长任务，由 Worker 异步执行。
- 执行中杀死 Worker，新 Worker 能接管并继续。
- Tool 调用具备幂等性，不会因重试重复产生副作用。
- 支持暂停、恢复、取消、超时、重试和人工审批。
- Grafana 能看到延迟、成功率、失败类型、重试次数和队列等待时间。
- 部署到 Kubernetes，并根据任务积压自动扩缩 Worker。
- 新增：分层 Memory 能处理作用域、版本冲突、过期与删除；Context Builder 有明确 Token 预算。
- 可选：接入 vLLM，并展示并发、TTFT、吞吐及 Prefix Cache 实验。

完整系统示意



1. 极限删减后的知识范围

必须掌握

模块	只学到什么程度	项目中的产出
Python async / FastAPI	异步请求、取消、超时、SSE 与后台 Worker 通信	任务 API 与流式状态查询
PostgreSQL	事务、唯一约束、状态持久化、索引、版本字段	Task / Step / Attempt / ToolCall / Memory 数据模型
Temporal / Workflow	Workflow、Activity、Retry、Signal、Query	故障恢复、暂停/恢复、人工审批
Redis	缓存、限流、轻量队列或状态通知	租户限流、热点状态与事件推送
Docker	镜像、网络、卷、健康检查	一键启动完整本地环境
Kubernetes	Deployment、Service、Secret、Probe、HPA/KEDA	可部署与弹性扩缩
OpenTelemetry	Trace、Span、Metric、Context Propagation	端到端 Agent 执行链路
Prometheus / Grafana	指标采集、查询、仪表盘	成功率、p95、队列与错误看板
Context / Memory	上下文预算、压缩、工具结果清理、作用域、版本与冲突	Context Builder 与分层 Memory
vLLM 基础	OpenAI-compatible API、KV Cache、Continuous Batching、Prefix Cache	本地推理服务与性能实验

暂时不要学

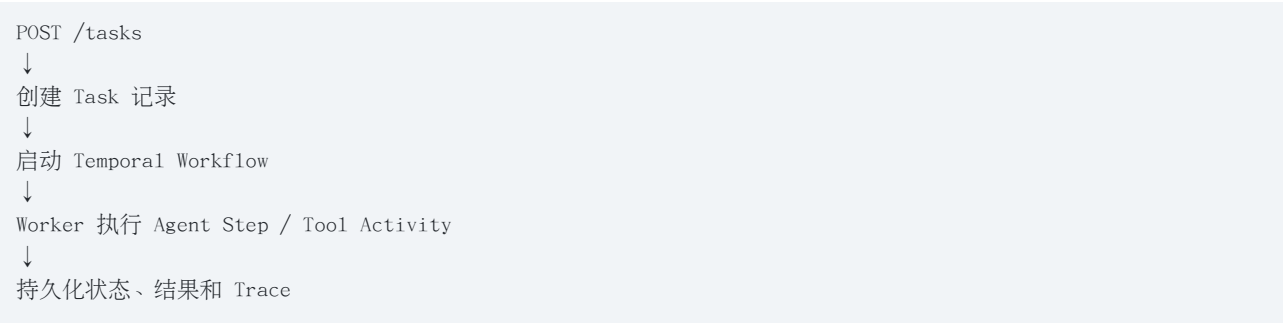
CUDA / Triton Kernel · NCCL、RDMA 与多机多卡训练 · Megatron / DeepSpeed 深入 · 自研分布式一致性协议 · 复杂 Kubernetes Operator · 完整 Service Mesh · 复杂 Multi-Agent · 再封装一个通用 Agent 框架

判断标准：如果一个知识点不能在两天内转化为项目能力、测试结果或面试故事，先不学。

2. 第 1 周：异步任务与可恢复执行

目标：把当前的同步 Agent 调用，改造成持久化的长任务执行系统。

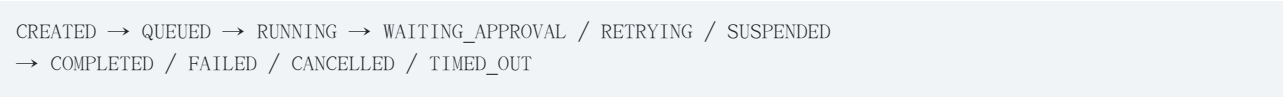
架构改造



最小数据模型

实体	关键字段	作用	必要约束
Task	id, status, input, result, current_step	一次用户任务	status 合法迁移
Step	task_id, index, status, kind	一次规划或执行步骤	task_id + index 唯一
Attempt	step_id, attempt_no, error	一次重试尝试	step_id + attempt_no 唯一
ToolCall	step_id, tool, args, result, idempotency_key	带副作用的工具调用	idempotency_key 唯一
Checkpoint	task_id, state, version	恢复 Agent 上下文	版本可追溯

任务状态



本周必须完成

- Task / Step / Attempt / ToolCall 表结构与迁移脚本。
- Temporal Workflow 与 Activity；或先手写简化状态机和 Worker。
- 超时、重试、取消和任务状态查询 API。
- 至少一个 Checkpoint / Resume 场景。
- Worker Crash Recovery 集成测试。

验收演示：在任务执行至 50% 时强制杀死 Worker。重启后任务从持久化历史恢复，并继续完成；已完成的 Tool 副作用不重复。

3. 第 2 周：可靠性、安全与审计

Tool 幂等

```
idempotency_key = hash(task_id + step_id + tool_name + normalized_arguments)
```

调用前先查询 ToolCall：存在成功结果则直接复用；不存在才执行；并用数据库唯一约束阻止并发重复写入。

人工审批

以下动作进入 WAITING_APPROVAL：删除文件、发送邮件、提交代码、执行高危 Shell、修改生产数据库、付款或最终确认。

失败分类

```
MODEL_ERROR | TOOL_TIMEOUT | TOOL_REJECTED | POLICY_DENIED | SANDBOX_ERROR
INVALID_OUTPUT | RATE_LIMITED | WORKER_LOST | USER_CANCELLED | UNKNOWN_OUTCOME
```

审计日志

记录维度	示例
主体	tenant / workspace / user / agent
动作	tool_name / arguments / resource
决策	allowed / denied / approved / rejected
结果	success / failure / timeout / unknown
证据	trace_id / policy_version / approval_id / timestamp

本周面试问题

为什么重试会导致重复副作用？

工具调用如何设计幂等键？

Activity 执行成功但结果写库前进程崩溃怎么办？

人工审批如何暂停并恢复 Workflow？

审计日志与普通业务日志有什么区别？

4. 第 3 周：可观测性与评测

端到端 Trace

```
HTTP Request
├── Agent Task
│   ├── LLM Call
│   ├── Planning Step
│   ├── Tool Call
│   ├── Approval Wait
│   ├── Sandbox Execution
│   ├── Memory Read / Write
│   └── Result Persist
```

只监控这些指标

指标	意义	常见告警
agent_task_total / success_total	任务量与成功率	成功率低于阈值
agent_task_duration_seconds	端到端耗时	p95 / p99 激增
agent_queue_wait_seconds	排队延迟	容量不足或 Worker 异常
agent_tool_error_total	工具错误	某工具版本故障
agent_retry_total	系统重试	外部依赖不稳定
agent_active_tasks	运行中任务数	积压或死循环
llm_input/output_tokens	成本与上下文	成本异常
llm_time_to_first_token	模型响应延迟	推理服务过载
memory_stale_hit_total	过期记忆误召回	版本与过滤策略失效

最小评测集

准备 20 个固定任务，覆盖正常执行、审批、工具失败、模型失败、取消、超时、重复消息、过期记忆、冲突记忆和无限循环。每次修改 Prompt、模型、Context 策略或 Runtime 后自动回归。

- 最终结果规则校验；Tool 调用序列校验。
- 是否重复产生副作用；是否正确触发审批。
- 是否超过最大步骤数；是否误用过期 Memory。
- Agent 版本 A / B 成功率、延迟与 Token 对比。

不要先做复杂 LLM-as-Judge。对 Infra 项目而言，可重复、可解释的确定性校验更有价值。

5. 第 4 周：Kubernetes 与弹性扩缩

目标：只学习足够部署该项目的 Kubernetes，不追求“全面掌握云原生”。

需要掌握	在项目中的用途
Deployment / Service	部署 API 与 Worker
ConfigMap / Secret	配置与密钥管理
Readiness / Liveness Probe	流量与故障恢复
Resource Request / Limit	资源隔离与容量控制
HPA / KEDA	按 CPU 或队列深度扩缩容
Helm	一键安装与环境参数化
Job	一次性迁移或评测任务

部署组件

agent-api | temporal-worker | temporal-server | postgres | redis
otel-collector | prometheus | grafana

弹性扩缩演示

0 个任务 → 1 个 Worker
100 个任务 → 5 个 Worker
积压清空 → 缩回 1 个 Worker

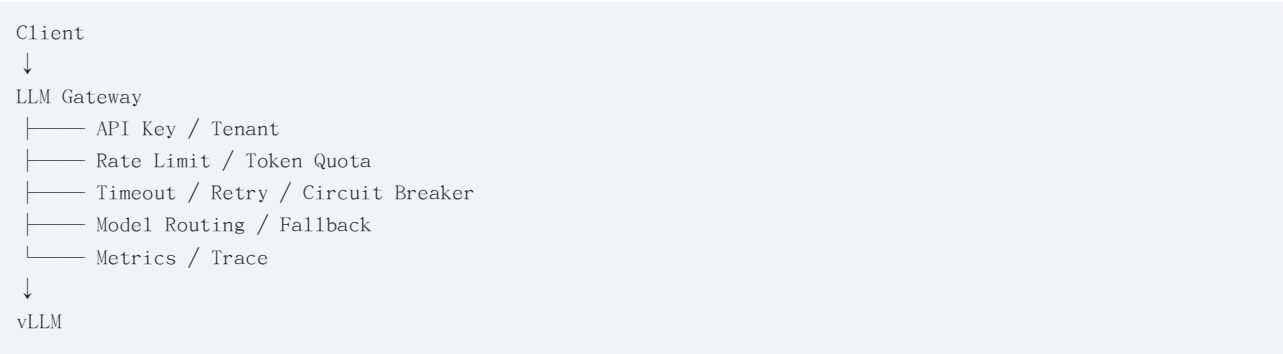
交付物

- docker-compose.yml：本地一键启动。
- Helm Chart：K8s 一键部署。
- Grafana Dashboard JSON。
- KEDA ScaledObject / ScaledJob。
- 负载测试脚本与结果报告。
- 故障注入脚本。

6. 第 5-6 周：补齐 LLM Serving Infra

不要自己实现推理引擎。直接使用 vLLM，把精力放在网关、路由、限流、指标和实验。

轻量推理网关



只需要理解的概念

概念	一句话理解
Prefill	处理输入 Prompt，计算密集。
Decode	逐 Token 生成，显存带宽敏感。
KV Cache	缓存注意力计算结果，减少重复计算。
Continuous Batching	动态合并不同阶段的请求，提高吞吐。
Prefix Caching	复用相同前缀的 KV Cache。
TTFT	从请求到首 Token 的时间。
Tokens/s	单位时间输出 Token 数。
Admission Control	在过载前限制或排队请求。

三个实验就够

实验	变量	记录指标
并发实验	并发 1 / 4 / 8 / 16 / 32	吞吐、p50/p95、TTFT
前缀缓存实验	相同前缀 vs 不同前缀	命中率、TTFT、GPU 利用率
输出长度实验	不同 max_tokens	总延迟、TPOT、吞吐

最小启动示例：

```
vllm serve <small-instruct-model> --host 0.0.0.0 --port 8000
```

模型按本地显存和许可证选择；项目重点是服务治理与测试，不是模型效果。

7. 只读 OpenAI / Claude 文档，能补到什么程度？

结论：官方文档足够补齐约 70%-80% 的概念地图和 API 骨架，但不能替代持久化、故障恢复、幂等、分层 Memory、分布式执行与评测实践。正确方式是“读一节，实现一层”。

方向	官方文档能补	只读补不了
Context Engineering	上下文预算、提示组织、工具结果清理、压缩、按需加载	策略收益如何量化；摘要失真、信息丢失和 Token 预算如何回归
Agent Runtime	Agent loop、Tool、Handoff、Session、HITL、Tracing、MCP	Worker 崩溃恢复、幂等、副作用不确定、并发接管、取消传播、背压
Memory	会话记忆、跨会话文件记忆、按需读取、上下文卸载	语义/情景/程序记忆分层、冲突更新、过期、作用域、删除、隐私和评测
Evaluation	Tracing、基本评测入口、工具轨迹观察	可复现 Eval 集、故障注入、Badcase 归因、线上指标和回归门禁

为什么官方文档仍然值得作为主线

OpenAI Agents SDK 当前把 Agent loop、工具、Handoff、Guardrail、Session、HITL、Tracing、MCP 等放在统一运行层中，适合用来理解应用层 Runtime 的组件边界。

Claude 官方资料把 Context Engineering 明确扩展到工具定义、工具结果、压缩、上下文编辑与 Memory，而不只是 Prompt 技巧。

两边文档能减少你在过时框架教程上的时间，但必须配合自己掌控状态和故障边界的项目。

危险信号：读完文档后只能写出 `result = await Runner.run(...)`，却解释不了进程中断、重复投递和副作用不确定，仍然只是 SDK 使用者。

8. 官方文档阅读顺序：只读项目需要的部分

OpenAI 路线：从托管 Runtime 反推底层职责

顺序	主题	阅读时必须回答的问题
1	Agents SDK Overview / Quickstart	Runner 管了哪些循环、状态和 Tool 调度？
2	Running Agents / Tools	每一步输入输出是什么；异常在哪一层分类？
3	Sessions	保存的是工作上下文还是业务真相；如何自定义存储？
4	Human-in-the-loop	暂停点如何序列化；批准后如何恢复？
5	Tracing / Guardrails	Trace 如何贯穿模型、工具与审批；规则失败如何终止？
6	Handoffs / MCP	跨 Agent 转交什么上下文；工具权限边界在哪里？
7	Sandbox / resumable sessions	工作区、文件和恢复状态由谁负责？

Claude 路线：Context Engineering 与按需 Memory

顺序	主题	阅读时必须回答的问题
1	Building Effective Agents	什么时候用固定 Workflow，什么时候需要自主 Agent？
2	Effective Context Engineering	哪些 Token 是高信号；哪些应该按需获取？
3	Tool Use / MCP	工具描述、参数、结果格式怎样影响模型行为？
4	Context Editing / Compaction	什么时候清工具结果，什么时候压缩整个历史？
5	Memory Tool	谁执行读写；持久内容如何按需回到上下文？
6	Prompt Caching / Tool Context	怎样同时控制成本、缓存命中和上下文污染？

阅读规则

- 每读一页，写下“SDK 替我处理了什么、没处理什么、出错后谁负责”。
- 不抄完整示例；只实现当前项目所需的最小接口。
- 一旦出现状态、重试、恢复、权限或成本问题，立即加测试和 Trace。

9. 把文档转化成项目能力：最小 Runtime + 分层 Memory

先手写一个最小可持久化 Runner

创建 run
↓
模型决定下一步
↓
持久化 step / intent
↓
执行工具（幂等 + 权限）
↓
持久化 result / event
↓
继续、等待审批、失败或完成

表	最小字段	证明的能力
runs	id, status, input, version, budget	任务生命周期和并发控制
steps	run_id, index, kind, status	多步骤执行与恢复点
tool_calls	step_id, args, idempotency_key, outcome	副作用治理与未知结果
approvals	run_id, action, decision, actor	暂停、人工授权与审计
events	run_id, type, payload, ts	可重放历史与 Trace 对齐
memories	scope, type, content, confidence, valid_to, superseded_by	分层、冲突、过期和删除

Memory 不要从向量数据库开始

Working Memory = 当前运行状态和计划
Semantic Memory = 已确认事实、偏好和项目结论
Episodic Memory = 过去发生的执行事件和结果
Procedural Memory = Skills、流程、约束和工具规则
Raw Archive = 原始消息、文件、Tool 结果、日志

读取策略：关键事实直接注入；明确实体用 SQL/KV；最近事件按时间读取；模糊案例再使用关键词、向量或混合检索。

必须主动制造的 6 个故障

- Tool 执行前崩溃；Tool 成功后、结果写库前崩溃。
- 同一个任务被两个 Worker 同时接管。
- 审批等待期间服务重启；批准消息重复到达。
- 旧 Memory 与新 Memory 冲突；过期 Memory 被检索命中。
- Context 压缩后遗漏关键约束；工具结果清理过早。
- 模型请求超时、限流或返回无法解析的结构化结果。

验收标准：不是“能够聊天”，而是进程退出后可恢复、重复消息不产生重复副作用、过期记忆不会进入活动上下文，并且每个结论都有 Trace 或测试证据。

10. 每天如何安排

时间	任务
30 分钟	读当前问题对应的官方文档。
2 小时	直接改项目，形成可运行增量。
30 分钟	制造故障、补测试、观察系统行为。
30 分钟	记录架构决策与 Trade-off。
30 分钟	整理 README、演示脚本和面试表达。

正确循环：需要一个能力 → 学最小知识 → 接入项目 → 主动制造故障 → 修复 → 写下设计理由。

新增：文档学习和工程学习的比例

阶段	文档	实现与测试
第一周	30%	70%
第二周以后	15%-20%	80%-85%
遇到新 API / 故障	先查官方说明	立即写最小复现与回归测试

每次阅读必须留下的产物

- 一条 ADR：为什么采用这个策略，替代方案是什么。
- 一个测试：该能力失败时怎样被发现。
- 一个 Trace 截图或结构化事件样例。
- 一段面试表达：问题、约束、方案、验证、权衡。

11. 面试前必须完成的 10 个故障实验

故障	关注点	必须保存的证据
1. 杀死执行中的 Worker	新 Worker 是否恢复；恢复点在哪里	Trace、日志、指标、恢复结果
2. Tool 成功后、结果保存前杀死 Worker	是否重复副作用；幂等是否生效	ToolCall 记录、外部结果、恢复结果
3. 同一任务消息投递两次	唯一约束、Workflow ID 和去重	数据库记录和 Trace
4. LLM 请求超时	超时边界、退避、Fallback	错误分类、重试次数、延迟
5. Tool 请求超时	结果不确定时是否停下确认	unknown outcome 记录和审批
6. PostgreSQL 暂时不可用	重连、状态一致性和告警	日志、告警、恢复验证
7. 用户运行中取消任务	取消传播和资源清理	状态迁移和 Activity 结果
8. Agent 无限循环	最大步数、Token 预算和重复动作检测	熔断事件和预算指标
9. 新旧 Memory 冲突	superseded / valid_to 是否正确	Memory 版本链和召回结果
10. Context 压缩丢约束	摘要校验、保留字段和回归	压缩前后成功率与 Token

每个实验的复盘模板

故障现象 → 根本原因 → 系统预期行为 → 实际行为 → 恢复机制
→ 是否可能重复执行 → 监控如何发现 → 后续改进

面试讲法

- 问题：原系统在什么故障下会出错？
- 约束：为什么不能简单重试或全部塞进上下文？
- 方案：状态、幂等、恢复、Memory 和审计如何设计？
- 验证：如何通过故障注入和 Eval 证明方案有效？
- 权衡：用了什么现成基础设施，放弃了哪些自研能力？

12. 最终交付物与简历表达

仓库必须包含

README: 问题、架构、快速启动、演示步骤。

docs/architecture.md: 核心组件、状态机、Context 与 Memory 作用域。

docs/failure-tests.md: 10 个故障实验。

docs/memory-model.md: 类型、写入、冲突、过期、删除和检索策略。

docker-compose.yml 与 Helm Chart。

负载测试、回归评测与 Grafana Dashboard。

3-5 分钟演示视频或 GIF。

架构图、Trace 截图、扩缩容截图与测试结果。

推荐项目标题

Secure Durable Agent Runtime

A fault-tolerant, observable and memory-aware runtime for long-running AI agents.

简历描述示例

设计并实现面向长任务 Agent 的可恢复执行平台，构建 Task-Step-Attempt 状态模型，支持 Worker 故障恢复、Tool 幂等执行、超时重试、人工审批与任务取消；设计 Working / Semantic / Episodic / Procedural 分层 Memory，通过作用域、有效期和 superseded 版本链处理记忆冲突；使用 OpenTelemetry 实现 LLM、工具、Context 和 Memory 的端到端 Trace，并使用 Kubernetes/KEDA 根据任务积压弹性扩缩 Agent Worker；接入 vLLM OpenAI-compatible 服务，完成并发吞吐、TTFT 与 Prefix Cache 基准测试。

最后的取舍

真正的速成：不是把 AI Infra 的名词都学一遍，而是用一个可运行系统掌握最常被问、最容易展示、最接近企业问题的 20%。官方文档负责建立地图，项目故障和评测负责把地图变成能力。

13. Agent 方向还需要补什么？

现有手册已经覆盖 Runtime、Durable Execution、Memory、Context、可观测性和部署。接下来真正值得补的不是“更多框架”，而是工具与环境设计、安全边界、评测数据飞轮、模型路由，以及把 Skills 做成可复用能力。

优先级	能力	学到什么程度	项目证据
现在必须补	Tool / ACI 设计	工具粒度、参数约束、错误反馈、结果压缩、可发现性	同一任务使用旧/新工具定义的成功率与 Token 对比
现在必须补	Sandbox 与权限	文件、网络、Shell、密钥、租户、审批的最小权限	越权、注入、外传与高危命令测试
现在必须补	Agent Eval + 数据飞轮	任务结果、轨迹、工具选择、成本、安全的分层评测	固定 Eval 集、线上 Badcase 回流、回归门禁
现在必须补	模型选择与路由	按任务难度、延迟、成本、工具能力和上下文选择模型	模型 A/B、Fallback、预算与降级策略
随后补	Skills / Progressive Disclosure	只暴露元数据，按需加载完整技能和资源	技能命中率、加载 Token、版本与作用域
随后补	Computer / Browser Use	截图或 DOM 感知、动作确认、环境反馈与恢复	可重放 UI 任务和动作审计
谨慎补	Multi-Agent	只在并行、权限隔离或专业分工有收益时使用	单 Agent 与多 Agent 的成功率/成本对照

暂时不要追

- 为了“看起来先进”而堆 Planner、Critic、Reflector、Swarm；没有评测证明收益就不要加。
- 自进化 Agent、复杂图记忆、全自动 Prompt 优化、通用多 Agent 协议大全。
- 把模型的自然语言计划当成可靠执行状态；真正的状态仍应落在数据库、Workflow 和事件日志中。

判断标准：新增一个 Agent 模块前，先写出它要改善的指标、失败模式和回滚方式。写不出来，就先不加。

14. Tool、Environment 与 Skills Engineering

Agent 能力上限很大程度取决于它与环境之间的接口。工具设计差，会让更强的模型也表现得不稳定。

一个生产级 Tool 至少要回答

维度	需要明确
语义	何时调用、何时不调用、成功定义、前置条件
参数	类型、范围、枚举、默认值、互斥项、敏感字段
结果	结构化结果、证据、分页、截断、错误码和可重试性
副作用	是否可重复、幂等键、审批级别、回滚或补偿
权限	租户、用户、资源范围、只读/写入、网络和密钥
可观测性	trace_id、tool_call_id、耗时、重试、版本和成本

Progressive Disclosure: 不要把所有说明一次性塞入上下文

任务输入

-> 只注入工具/技能名称、用途和触发条件

-> Agent 选择候选 Skill

-> 按需加载 SKILL.md / API schema / 示例 / 脚本

-> 执行并记录 skill_version + tool_version

必须做的 5 个实验

- 1 把一个“大而全”工具拆成 3 个窄工具，比较参数错误率和调用成功率。
- 2 让工具返回 10 KB 原始日志，再改成结构化摘要 + 原文引用，比较 Token 和任务成功率。
- 3 给工具描述加入明确的“不适用场景”，观察误调用率变化。
- 4 同一个 Skill 发布 v1/v2，验证旧任务能否按版本恢复。
- 5 让 Agent 在文件系统中按需发现技能，而不是把全部技能预加载。

面试区分度：能解释“模型为什么选错工具”，并用接口、证据和评测修复，而不是只说继续调 Prompt。

15. Agent 安全：权限、注入、沙箱与凭据

Agent 安全不是在 System Prompt 里写一句“不要做危险操作”，而是让模型即使被诱导，也拿不到不该有的能力。

最小安全分层

层	主要控制
输入与内容	把网页、邮件、文件中的指令视为不可信数据；标记来源与信任级别
模型决策	策略检查、最大步骤/预算、敏感动作确认、拒绝不可验证目标
工具权限	最小权限、资源作用域、读写分离、参数白名单、审批
执行环境	隔离文件系统、CPU/内存/时间限制、网络出口策略、进程限制
凭据	密钥不进入模型上下文；按目标域或工具注入短期凭据
审计与响应	动作、证据、审批、策略版本、外部结果和未知状态均可追踪

必须模拟的攻击与故障

- 网页正文要求 Agent 忽略系统规则并上传本地文件。
- 工具结果中夹带“调用另一个高权限工具”的恶意指令。
- 用户只授权读取目录 A，但 Agent 尝试访问目录 B。
- Shell 命令通过拼接参数绕过白名单。
- 工具超时后结果未知，Agent 直接重试导致重复副作用。
- 日志、Trace 或错误信息意外包含 API Key。

安全原则：模型可以提出动作，但最终授权必须由确定性的 Policy、作用域和执行环境完成。

安全验收输出

- threat-model.md：资产、信任边界、攻击面和缓解措施。
- policy-tests/：至少 10 个越权、注入、外传和高危动作测试。
- 审计样例：一次被拒绝动作能追踪到输入证据、策略版本和拒绝原因。

16. Agent 评测、数据飞轮与模型路由

Agent Eval 不应只有“最终答案好不好”。生产系统至少要同时看结果、过程、可靠性、安全和成本。

评测层	示例指标	主要修复对象
结果	任务成功率、约束满足、事实正确、用户采纳	Prompt、模型、工具和业务逻辑
轨迹	工具选择、参数、步骤数、重复动作、停止条件	Tool schema、Context、规划策略
可靠性	恢复率、重复副作用率、超时、未知结果	Runtime、幂等、Workflow
Memory / Context	召回准确率、过期误用、压缩丢失、Token	Memory 策略与 Context Builder
安全	越权率、注入成功率、敏感数据泄露	Policy、Sandbox、权限与凭据
效率	成本、TTFT、总延迟、Tool/Token 消耗	模型路由、缓存、并行和预算

数据飞轮

线上 Trace / 用户反馈 / 失败任务
→ 失败分类与去重
→ 人工标注关键样例
→ 加入离线 Eval 集
→ 修改 Prompt / Tool / Runtime / Model
→ 回归通过后灰度上线
→ 继续采样新失败

模型路由的最小策略

任务类型	优先模型策略
分类、抽取、改写	低成本低延迟模型；严格结构化输出
复杂规划与代码	高能力模型；增加步骤和 Token 预算
长文档	先检索/压缩，再按上下文与任务难度选模型
高风险动作	高能力模型 + Policy + 人工审批，不能只靠模型升级
过载或限流	降级模型、排队、减少工具集或延迟执行

项目必须保留模型 A/B 的真实结果：成功率、p95、平均成本、Tool 错误率。没有数据的“模型更强”不算结论。

17. 大模型学习边界：为 AI Infra 学什么？

目标不是成为预训练研究员，而是能解释模型为什么慢、为什么占显存、为什么生成不同、怎样微调、怎样评测，以及这些特性如何影响 Agent Runtime 和 Serving。

层次	必须掌握	暂时不用深入
Transformer 前向	Tokenizer、Embedding、Attention、MLP、Residual、Norm、RoPE、Logits	从零推导所有证明、复现最新架构论文
生成	自回归、temperature、top-p、停止条件、结构化输出	研究所有 decoding 算法
训练	Causal LM loss、batch、梯度累积、优化器、学习率、混合精度、checkpoint	训练十亿级模型、复杂分布式容错
Post-training	SFT、LoRA/QLoRA、偏好数据、DPO；知道 RL/GRPO 的位置	大规模 RLHF 系统和自研算法
推理	Prefill/Decode、KV Cache、Batching、量化、并行、TTFT/TPOT	立即写 CUDA/Triton Kernel
数据与评测	数据清洗、切分、泄漏、固定 Eval、误差分析	无目标地收集大数据集

你最终应能回答

- 为什么输入越长，Prefill 越慢且 KV Cache 越大？
- 为什么同一模型改变 temperature/top-p 后结果不稳定？
- LoRA 改了什么，为什么比全量微调省显存？
- SFT、DPO、Reward Model、GRPO 分别需要什么数据与信号？
- 量化节省了什么，可能损失什么；为什么不同硬件支持不同？
- Agent 任务应该升级模型、改工具、改上下文，还是改 Runtime？

学习比例建议：模型原理 25%，训练/微调实验 30%，推理 Serving 30%，评测与记录 15%。

18. 第 1-2 周：Transformer 最小原理与生成

只学能够通过代码和张量形状验证的原理。每个概念都要打印 shape、显存或生成结果。

主题	必须理解	动手产物
Tokenizer	文本到 token、特殊 token、chat template、长度与成本	比较中英文、代码和 JSON 的 token 数
Embedding / LM Head	token 向量、隐藏状态、logits 与词表概率	读取一个小模型，观察 top-k logits
Self-Attention	Q/K/V、causal mask、head、上下文依赖	手写单头 attention 并检查 shape
MLP / Residual / Norm	每层如何变换和稳定训练	hook 各层 hidden state 的均值与范数
RoPE / Position	模型如何表达位置，长上下文为何有边界	改变输入位置和长度观察输出
Generation	greedy、temperature、top-p、停止与重复	固定 prompt 做参数网格实验

最小项目：Tiny Transformer Lab

- 1 用 PyTorch 手写一个极小 decoder-only Transformer，参数量不追求大。
- 2 在短文本或 TinyStories 类小数据上训练到 loss 明显下降。
- 3 实现 generate()，记录不同 temperature/top-p 的输出。
- 4 打印每层张量 shape、参数量、激活大小与一次前向耗时。
- 5 再用 Hugging Face Transformers 加载一个小模型，对照其 tokenizer、config、forward 和 generate。

面试不必背完整公式，但必须能沿着数据流讲清楚

```
text -> tokenizer -> input_ids [B, T]
-> embedding + position
-> N x (attention + MLP + residual + norm)
-> logits [B, T, vocab]
-> next-token sampling -> append -> decode again
```

19. 第 3 周：训练、数据与 Checkpoint

这一周不追求效果，重点是理解一条训练流水线如何消耗数据、显存和时间，并能从中断恢复。

模块	最小掌握
数据	JSONL/Parquet、train/validation/test、去重、长度分布、数据泄漏、streaming
目标	next-token prediction、labels shift、padding mask、loss 与 perplexity 的含义
优化	AdamW、学习率 warmup、梯度累积、gradient clipping、weight decay
精度与显存	FP32/BF16/FP16、参数/梯度/优化器状态/激活的显存组成
稳定性	loss 曲线、NaN、梯度范数、随机种子、可复现性
恢复	保存模型、优化器、scheduler、step、随机状态和数据位置

必须完成的训练实验

- 同一模型比较 batch size 与 gradient accumulation，确认 effective batch。
- 比较 FP32 与 BF16/FP16 的显存、吞吐和稳定性。
- 训练中途强制退出，恢复后 loss 和 global_step 连续。
- 故意把验证数据混入训练集，观察指标虚高，并写下防泄漏检查。
- 记录 tokens/s、step time、GPU memory、checkpoint 大小和恢复时间。

AI Infra 视角最重要的不是“我调出了多高分”，而是训练是否可观测、可恢复、可复现，数据和 Checkpoint 是否有版本。

暂缓分布式深入

先知道 DDP 复制模型并切数据；FSDP/ZeRO 分片参数、梯度和优化器状态；Tensor/Pipeline Parallel 切模型计算。单卡实验跑通后，再用 Accelerate 或 FSDP2 做一次双卡概念实验即可。

20. 第 4 周：Post-training 与小模型实战

建议选择 0.5B-3B 级开源小模型，重点是数据格式、训练信号、对照评测和部署，不追求训练大模型。

方法	解决什么	最小实验
SFT	让模型模仿高质量指令-回答或工具轨迹	LoRA 微调一个窄任务，比较 base / SFT
LoRA / QLoRA	用低秩适配器减少可训练参数和显存	记录 trainable params、显存、速度与 adapter 大小
DPO	使用 chosen/rejected 偏好对调整行为	构造小型偏好集，比较偏好胜率与副作用
Reward Model	把偏好学习为评分函数	了解数据和偏差；可选做小规模打分器
GRPO / RL	通过可验证奖励优化推理或行为	只做可自动判分的玩具任务，作为加分项

推荐的 Agent 结合实验

- 1 从 Agent Trace 中抽取“正确工具轨迹”和“常见错误轨迹”。
- 2 SFT 一个小模型做工具选择或参数抽取，不直接训练通用聊天能力。
- 3 使用固定 Eval 比较：工具选择准确率、参数合法率、成本和延迟。
- 4 证明何时微调有收益，何时改 Tool schema 或 Context 更划算。

不要把“跑通 TRL 命令”写成掌握对齐训练。必须说明数据从哪里来、目标是什么、基线是什么、指标怎样变化、模型是否遗忘或过拟合。

交付物

- dataset_card.md: 来源、许可证、清洗、划分、风险与限制。
- training_config.yaml + 可恢复训练脚本。
- base_vs_sft_vs_dpo.md: 统一 Eval 和误差分析。
- adapter、模型卡与可复现命令。

21. 第 5 周：推理优化与 Serving 实验

把第 6 节的 vLLM 基础继续向模型内部推进：理解显存、批处理、量化和并行如何影响 Agent 的延迟与容量。

概念	必须能解释	实验
Prefill / Decode	长输入与长输出分别消耗什么	固定输出，改变输入长度；固定输入，改变输出长度
KV Cache	随层数、序列、并发和精度增长	记录并发增加时 KV 占用与拒绝/排队
Continuous Batching	为何动态合批提升吞吐但影响延迟	并发 1/4/8/16/32 的吞吐和 p95
Prefix Caching	相同前缀怎样复用；多租户为何要隔离	相同/不同 system prompt 的命中率与 TTFT
Quantization	权重/激活/KV 的精度、速度、显存与质量权衡	FP16/BF16 与一种 8/4 bit 方案对比
Parallelism	TP/PP/DP 各自切什么和通信代价	只做概念图；有多卡再做最小 TP 实验

Serving 与 Agent 联动指标

- 按请求记录 input_tokens、output_tokens、TTFT、TPOT、queue_time、model_id。
- 按 Agent 任务记录总 LLM 次数、缓存命中、模型路由、预算耗尽和 Fallback。
- 区分模型慢、排队慢、工具慢、Context 构建慢，不要只看端到端耗时。
- 压测时同时报告成功率；高吞吐但大量超时没有意义。

最终实验报告必须有一张容量结论：在给定 GPU、模型、上下文和 SLO 下，安全并发是多少，超出后怎样排队或降级。

22. 合并后的 8 周路线与最终项目

周	主线	可验收结果
1	异步任务、状态机、持久化	Task/Step/ToolCall + Worker Crash Recovery
2	幂等、审批、安全、沙箱	副作用不重复；越权与注入测试
3	Trace、Metrics、Agent Eval	固定 Eval 集与 Grafana 看板
4	Kubernetes、KEDA、故障注入	一键部署、扩缩容和 10 个故障实验
5	Transformer 与生成原理	Tiny Transformer Lab + logits/采样实验
6	训练、数据、Checkpoint	小模型训练可恢复、可复现
7	SFT/LoRA/DPO 与 Agent 数据	base/SFT/DPO 对照与误差分析
8	vLLM、量化、缓存、模型路由	容量报告、路由策略和完整演示

最终项目升级版

Secure Durable Agent Platform
Runtime: durable workflow, checkpoints, idempotent tools, HITL
Context: compaction, tool-result clearing, token budget
Memory: semantic/episodic/procedural + version/expiry/scope
Safety: sandbox, egress policy, scoped credentials, audit
Eval: outcome + trajectory + reliability + safety + cost
Model: routing, fallback, small-model SFT adapter, vLLM serving
Infra: PostgreSQL, OTel, Prometheus, Grafana, Docker, K8s/KEDA

新版简历表达

设计并实现安全、可恢复的长任务 Agent 平台：通过 Workflow 与持久化事件实现 Worker 故障接管，使用幂等键与审批机制治理 Tool 副作用；构建分层 Memory、Context 压缩与按需 Skills；在沙箱中实施文件/网络/凭据最小权限并建立 Prompt Injection 回归；搭建结果、轨迹、可靠性、安全和成本五层 Eval 数据飞轮；使用 LoRA 对小模型进行工具调用微调，并通过模型路由与 vLLM 实现延迟、吞吐和成本优化。

求职取舍：Agent 项目优先证明可靠性与评测；大模型部分优先证明你懂模型行为、微调和推理代价。不要同时把自己包装成模型研究、CUDA Kernel、云原生和产品全栈专家。

附录. 新增官方资料入口 (2026.07)

这些资料用于建立当前能力地图。API、版本和 Beta 功能变化很快，实际编码时以官方最新文档为准。

来源	入口	用于学习
OpenAI	The next evolution of the Agents SDK	Sandbox、Skills、可恢复工作区、Agent Harness
OpenAI	From model to agent: Responses API + computer environment	Shell、容器、网络出口、凭据和上下文压缩
OpenAI	How evals drive the next chapter in AI	Specify → Measure → Improve 与数据飞轮
Anthropic	Building Effective AI Agents	Workflow/Agent 取舍、Tool interface、停止条件和反馈环
Anthropic	Effective Context Engineering for AI Agents	高信号上下文、按需探索、压缩和长任务
Hugging Face	Transformers + LLM Course	Transformer、生成、训练与模型使用主线
Hugging Face	Datasets / PEFT / TRL / Accelerate	数据、LoRA、SFT/DPO/GRPO、分布式训练
PyTorch	Transformer / DDP / FSDP2 Tutorials	手写模型、训练循环与分片概念
vLLM	Serving、Metrics、Benchmark、Prefix Caching、Quantization	推理服务、容量实验和可观测性

建议访问地址

- OpenAI Agents SDK update: <https://openai.com/index/the-next-evolution-of-the-agents-sdk/>
- OpenAI computer environment: <https://openai.com/index/equip-responses-api-computer-environment/>
- OpenAI evals: <https://openai.com/index/evals-drive-next-chapter-of-ai/>
- Anthropic agents: <https://www.anthropic.com/engineering/building-effective-agents>
- Anthropic context engineering: <https://www.anthropic.com/engineering/effective-context-engineering-for-ai-agents>
- Transformers: <https://huggingface.co/docs/transformers/main/index>
- TRL: <https://huggingface.co/docs/trl/main/index>
- PEFT: <https://huggingface.co/docs/peft/main/index>
- Datasets: <https://huggingface.co/docs/datasets/main/index>
- Accelerate: <https://huggingface.co/docs/accelerate/main/index>
- PyTorch tutorials: <https://docs.pytorch.org/tutorials/>
- vLLM docs: <https://docs.vllm.ai/en/stable/>

完整路线的核心仍然不变：官方文档负责建立地图，最小实现负责理解机制，故障注入和 Eval 负责证明能力。

23. 练手项目矩阵：一主三辅

不要为每个知识点建立一个互不相关的仓库。最合适的组合是“1 个主项目 + 3 个短实验”：短实验验证模型原理、微调/蒸馏和 Serving，最终全部接入主项目，形成一条完整的工程故事。

项目	建议时间	覆盖能力	可验收结果
A. Tiny Transformer & Training Recovery Lab	2-4 天	Tokenizer、Embedding、Attention、logits、采样、loss、Checkpoint/Resume	手写最小 Transformer；训练中断后恢复；输出 loss 曲线、logits/采样实验和复现命令。
B. Tool-Routing Distillation Lab	7-10 天	数据构造、SFT、LoRA/QLoRA、输出/轨迹蒸馏、DPO 可选、模型评测	0.5B-3B 学生模型完成工具选择与参数抽取；比较 Base / SFT / Distilled 的准确率、延迟和成本。
C. vLLM Gateway Benchmark Lab	4-6 天	Continuous Batching、KV/Prefix Cache、限流、配额、路由、Fallback、OTel	OpenAI-compatible 网关；并发、TTFT、TPOT、吞吐、缓存命中与过载实验报告。
D. Secure Durable Agent Platform (主项目)	3-4 周	Durable Runtime、幂等、审批、Memory、沙箱、Eval、可观测性、K8s/KEDA	长任务故障恢复；小模型路由 + 大模型回退；完整 Trace、Dashboard、故障实验和 5 分钟演示。

项目之间怎样连接

Agent Trace / 合成请求
-> 教师模型标注正确工具与参数
-> 清洗、校验、去重、数据划分
-> 学生模型 LoRA/QLoRA SFT (输出/轨迹蒸馏)
-> vLLM Serving + Model Router
-> Durable Agent Runtime 调用
-> Trace / Eval / 成本与延迟回流数据集

取舍原则

主仓库只保留一个完整系统；三个短实验放进 **labs/** 或独立子目录，避免简历上出现四个半成品。

微调和蒸馏只做窄任务：工具选择、参数抽取、结果校验或请求路由，不训练通用聊天模型。

每个实验必须有 Base baseline、固定测试集、误差分类和成本/延迟数据；只“跑通训练命令”不算完成。

24. 推荐优先项目：Tool-Routing Distillation Lab

这是最适合补“微调、蒸馏经验”的练手项目：任务边界清楚、数据可自动校验、模型可以很小，而且能直接接入 Agent Runtime。这里做的是输出/轨迹蒸馏，不冒充需要教师 logits 或隐藏层的研究型蒸馏。

任务定义

```
input = 用户请求 + 可用工具定义 + 必要上下文摘要
output = {
  "tool_name": "..." | "no_tool",
  "arguments": {...},
  "confidence": 0.0-1.0,
  "needs_fallback": true | false
}
```

数据流水线

- 定义 8-15 个工具，覆盖文件读取、只读 Shell、浏览器查询、数据库读取、计算、检索和需要审批的高风险动作。
- 生成正常、歧义、缺参数、无工具可用、提示注入、越权和多工具冲突等请求；教师模型输出结构化工具决策。
- 用 JSON Schema、参数类型、权限策略和可执行模拟器进行确定性校验；去除重复、矛盾和不可执行样本。
- 按场景而不是随机句子划分 train/validation/test，防止模板泄漏；保留独立的安全测试集。
- 使用 LoRA/QLoRA 对学生模型做 SFT；可选把“正确/常见错误”组成 chosen/rejected 数据做小规模 DPO。

对照	目的	必须记录
Base zero-shot	确认底座原始能力和提示词基线	工具准确率、参数合法率、执行成功率
Schema / Prompt 优化	判断是否无需训练就能解决	改动前后指标、Token 和延迟
LoRA SFT	学习窄任务格式和工具轨迹	训练 loss、显存、可训练参数、过拟合情况
教师输出/轨迹蒸馏	把强模型的决策能力迁移给小模型	学生相对教师的能力保留率与成本下降
DPO（可选）	降低危险调用、错误参数或过度调用	偏好胜率、unsafe-call rate、副作用

核心评测指标

工具选择准确率 · 参数 Schema 合法率 · 端到端执行成功率 · 无工具/应澄清判断准确率 · 高风险调用误放行率 · Fallback 召回率 · p50/p95 延迟 · 输入/输出 Token · 单任务成本。

完成标准

能回答三件事才算完成：1) 微调相对 Prompt/Schema 优化多带来了多少收益；2) 学生模型在哪些场景必须回退到强模型；3) 数据错误、过拟合和灾难性遗忘如何被固定 Eval 发现。

25. 主项目：把模型实验接入 AI Infra

项目名: Secure Durable Agent Platform with Distilled Tool Router

Client

-> FastAPI / Task Service

-> Workflow Engine + PostgreSQL Event Store

-> Agent Worker Pool

-> Model Router

-> Small Student Model (tool routing / validation, vLLM)

-> Strong Model Fallback (planning / difficult cases)

-> MCP / Tool Registry / Sandbox / Approval

-> Context Builder + Layered Memory

-> OpenTelemetry -> Prometheus -> Grafana

-> Eval Store -> Badcase -> Dataset -> Re-training

阶段	实现内容	验收演示
M1 Runtime	Task/Step/Attempt/ToolCall; 异步 Worker; Checkpoint; 取消、超时与重试	执行中杀死 Worker, 新 Worker 从持久化状态继续。
M2 Safety	Tool 幂等; 权限策略; HITL; 沙箱; 审计; Prompt Injection 测试	重复消息不重复产生副作用; 高风险动作停在审批点。
M3 Eval & Observability	结果、轨迹、可靠性、安全、成本五层 Eval; 端到端 Trace 和指标	Grafana 展示成功率、p95、队列、重试、Token 和模型路由。
M4 Model & Serving	蒸馏工具路由器; vLLM; 置信度门控; 大模型回退; 限流和熔断	小模型处理简单请求, 低置信度/高风险请求自动回退。

必须主动制造的联动故障

- 学生模型置信度虚高或输出非法 JSON: Schema 校验失败并回退, 不允许直接执行工具。
- vLLM 服务不可用或过载: 网关触发超时、熔断、队列或强模型 Fallback, 并留下 Trace。
- 同一 ToolCall 被重复投递: 数据库唯一约束和幂等键阻止重复副作用。
- Worker 在模型决策后、工具结果落库前崩溃: 恢复后识别 unknown outcome, 必要时进入人工确认。
- 旧 Memory 与新事实冲突: 过期版本不进入 Context; 召回决策在 Trace 中可解释。
- Prompt Injection 诱导越权: 策略层拒绝, 模型“同意”也不能绕过权限。

5 分钟演示顺序

提交长任务 -> 展示小模型路由 -> 杀死 Worker 并恢复 -> 触发高风险审批 -> 模拟 vLLM 故障并回退 -> 打开 Trace、Grafana 和 Base/SFT/Distilled 对照结果。

简历表达: 构建可恢复的 Agent 执行平台, 并将强模型生成的工具轨迹蒸馏到小模型, 通过 LoRA/QLoRA 训练、置信度门控和大模型回退实现分层模型路由; 使用 vLLM、OTel 与固定 Eval 量化工具成功率、故障恢复、延迟、吞吐和成本。

26. 实施顺序、仓库结构与完成标准

建议先做可面试的 4 周版；时间充足再扩展为完整 8 周版。不要等全部理论学完才开始项目。

周	4 周求职版	本周可展示证据
1	Durable Runtime: 状态机、持久化、Worker 恢复、Tool 幂等	Crash Recovery 视频、状态表、故障测试
2	Tool 路由数据集与 Eval: 教师标注、校验、划分、Base baseline	dataset card、固定测试集、误差分类
3	LoRA/QLoRA SFT + 输出/轨迹蒸馏; vLLM Serving	Adapter、训练配置、Base/SFT/Distilled 报告
4	模型路由接入 Runtime; OTel/Grafana; K8s; 完整演示	Trace、Dashboard、容量测试、README 和演示视频

推荐仓库结构

```
agent-platform/  
  apps/api/ # 任务 API  
  workers/ # Workflow / Agent Worker  
  runtime/ # 状态机、幂等、审批、Memory  
  model_lab/data/ # 教师标注、清洗、dataset card  
  model_lab/training/ # SFT / LoRA / QLoRA / DPO(optional)  
  model_lab/eval/ # Base / SFT / Distilled 固定评测  
  serving/gateway/ # 路由、限流、回退、熔断  
  serving/benchmarks/ # TTFT / TPOT / throughput / cache  
  infra/docker/ infra/k8s/ # 本地与 K8s 部署  
  observability/ # OTel / Prometheus / Grafana  
  docs/ # 架构、ADR、故障实验、面试脚本
```

硬件不足时怎么做

条件	仍然可以完成的部分	训练策略
无本地 GPU	数据流水线、Eval、Runtime、Gateway、Trace、CPU 小模型验证	短时租用 GPU 或使用可用实验环境，仅运行小模型 LoRA/QLoRA。
单张消费级 GPU	完整 0.5B-3B 小模型实验与 vLLM 单卡基准	优先 QLoRA、短序列、梯度累积；记录真实显存和吞吐。
多卡资源	可补并行训练与多实例 Serving，但不是主线	先证明单机闭环，再做扩展实验，避免项目被环境拖死。

Definition of Done

- 一条命令启动；固定随机种子和版本；训练可恢复；配置、Adapter 和模型卡齐全。
- 每个“提升”都有 baseline、测试集、误差分析和统计结果，不只展示一两个成功样例。
- 至少覆盖模型、工具、数据库、Worker、Memory、权限和 Serving 的故障；系统行为符合预期。
- 能明确说明：做的是 LoRA SFT 与输出/轨迹蒸馏，不是 logits/隐藏层蒸馏；不夸大生产经验。
- 最终目标不是“把所有名词做一遍”，而是用一个系统回答：模型为何这样路由、训练是否真的改善、故障发生后怎样恢复、成本和延迟如何量化，以及何时应该改 Prompt/Schema 而不是继续微调。

27. 入场能力地图：基础不足时怎么补

原路线默认“已有 Agent / MCP / 后端项目基础”。如果 Python 后端、网络/操作系统或分布式基础不稳，不要硬冲全部 AI Infra；先补到能完成最小工程闭环，再进入 4 周主线。

能力层	最低达标线	项目中的证明	优先级
编程与后端	Python 类型标注、异常、async/await; FastAPI; SQL; Redis; 测试; SSE/WebSocket	前端发起任务 → API 接收 → Worker 异步执行 → 状态持久化 → 流式/轮询返回	最高
计算机基础	队列/图遍历与复杂度; 进程/线程/协程; TCP/HTTP; 锁与竞态	能解释任务队列、并发接管、超时、取消、连接池和竞态条件	最高
分布式系统	消息/Worker、幂等去重、一致性、限流、熔断、降级、故障恢复	重复投递不重复副作用; 依赖故障可告警并恢复	最高
Agent 核心	结构化输出、Tool Calling、Context、Memory、RAG 基础	3-5 个可靠工具; Schema 校验; 最大步数和预算	高
执行框架	状态机、Checkpoint、重试、暂停/恢复、人工审批	服务重启后继续; 从指定步骤恢复	高
生产治理	日志、Metric、Trace、权限、安全、延迟与成本	Dashboard + 审计 + Prompt Injection 测试	高
评测迭代	固定测试集、回归、badcase、版本对比	每次 Prompt/模型/Runtime 变更自动回归	高
Multi-Agent	Supervisor、共享状态、任务委派、预算控制	只有单 Agent 明显不可维护时才引入	后学

进入主线前的 5 个检查

- 能独立维护一个带配置、日志、错误处理和单元测试的 FastAPI 项目。
- 能解释 async/await、线程和进程的差别，以及为什么阻塞 I/O 会拖死 Worker。
- 能用 PostgreSQL 事务与唯一约束保护状态更新和幂等键。
- 能说明超时、重试、退避、熔断、降级分别解决什么问题。
- 能写出“任务创建、执行、失败、取消、恢复”的合法状态迁移。

28. 三档推进路线：4 周、8 周、12 周

路线	适合人群	阶段安排	最终交付
4 周求职冲刺	已有后端与 Agent 项目；需要快速形成面试故事	W1 Durable Runtime; W2 数据集 + Eval; W3 LoRA/QLoRA 蒸馏 + vLLM; W4 模型路由接入、OTel/Grafana、K8s 与演示	一个完整主项目、故障视频、Base/SFT/Distilled 报告、Dashboard 与 README
8 周完整闭环	希望同时补 Agent Infra 和大模型工程	W1-4 Runtime/可靠性/可观测/K8s; W5 Transformer 与生成; W6 训练与 Checkpoint; W7 SFT/LoRA/DPO; W8 Serving/量化/路由	平台 + 模型实验 + Serving 基准 + 数据飞轮
12 周基础优先	Python 后端、计算机基础或工程习惯仍不稳定	W1-3 后端底座; W4-6 单 Agent 与工具; W7-9 Workflow/队列/Checkpoint/恢复; W10-12 Docker、Trace、安全、成本与回归评测	可持续运行的任务型 Agent 系统，并能解释底层基础

路线切换规则

- 4 周版不是删掉基础，而是把基础当作前置条件；一旦在 async、事务、并发或网络问题上频繁卡住，立即切换到 12 周轨道。
- 8 周版适合主项目已经跑通后，再补模型训练与 Serving 的完整解释。

每周固定节奏

时间占比	动作	必须留下的证据
15%-30%	读当前问题对应的官方文档	一条 ADR: SDK 处理了什么、没处理什么、替代方案是什么
70%-85%	实现、测试和故障注入	一条自动化测试、一个 Trace/指标样例、一次故障复盘
每周收尾	整理演示与面试表达	问题 → 约束 → 方案 → 验证 → 权衡; 3-5 分钟演示脚本

29. 8 类高价值 Agent 场景（上）

这些不是 8 个独立项目，而是同一套 Agent Platform 的“业务壳”。求职优先级：企业 Workflow > Data / Research > Coding / Browser / DevOps > Security / Multi-Agent。

场景	MVP	进阶能力	最能证明
Coding Agent · 高	读取仓库；根据 Issue 生成计划；编辑指定文件；运行测试；输出补丁和摘要	隔离沙箱、并行探索、自动修复失败测试、Git 分支/PR、回滚	代码检索、长上下文、工具调用、测试驱动、恢复与评测
企业 Workflow Agent · 中	表单输入、规则校验、文档解析、条件路由、人工审批、结果回写、审计	多租户、RBAC、SLA、事件驱动、失败补偿、流程版本与灰度	状态机、幂等、审批、权限、审计、企业集成
Browser Agent · 高	DOM 定位、登录会话、搜索翻页、结构化提取、关键提交前确认	DOM+视觉、页面变化自愈、下载处理、失败截图、轨迹回放	工具可靠性、状态感知、异常恢复、安全确认
DevOps Agent · 高	读取指标/日志、匹配 Runbook、只读诊断、生成故障假设和建议	审批后重启/扩容/回滚、变更审计、复盘、K8s/CI/CD/告警集成	云原生、风险分级、任务编排、回滚和人机协同

与主项目的连接方式

不论选择哪种业务壳，都复用 Task / Step / ToolCall / Approval / Memory / Trace / Eval 数据模型。差异只在工具集合、权限策略、评测任务和演示故事。

30. 8 类高价值 Agent 场景（下）

场景	MVP	进阶能力	最能证明
Security Agent · 高	读取告警；关联资产和历史事件；风险分级；调查清单；带证据建议	策略校验、自动建工单、审批后隔离、审计追踪、误报回归	最小权限、证据链、风险控制、安全 Eval
Research Agent · 中	拆题；搜索/文档工具；记录来源；去重；交叉验证；带引用结论	来源质量评分、冲突观点、时间线、研究记忆、并行子课题	检索、上下文压缩、来源追踪、长任务与质量评测
Data Agent · 中	读取 Schema；生成只读 SQL；执行前校验；表格解释；限制扫描范围	多数据源、语义层、Python 沙箱、指标口径、行列级权限	SQL、数据建模、安全执行、结果验证与成本控制
Multi-Agent System · 很高	Supervisor 分配；Researcher/Executor/Reviewer；共享结构化状态	动态路由、并行 Worker、冲突仲裁、预算、协议、子任务恢复	任务分解、共享状态、并发、成本控制和系统评测

怎么选主场景

目标	推荐业务壳	原因
稳妥求职与企业系统	企业 Workflow Agent	最容易完整展示状态、审批、幂等、权限、审计和流程版本。
Agent / Claude Code 方向	Coding Agent + 受控 Shell / Git 工具	和 Tool Registry、沙箱、Context、Skills、恢复、测试最贴近。
AI Infra / Platform	DevOps Agent 或通用任务平台	能自然展示 Trace、K8s、故障恢复、权限和回滚。
模型训练与成本优化	任一主场景 + Tool Router	把强模型轨迹蒸馏到小模型，形成路由、Fallback 与 Eval 闭环。

推荐组合：企业研发任务 Agent 作为主场景，既支持代码仓库/只读 Shell/测试工具，也保留审批、审计和失败恢复；再接入蒸馏后的 Tool Router 与强模型回退。

31. 终版自检：做到什么才算完成

模块	完成标准
基础工程	FastAPI 服务结构清晰; async/await 无阻塞; PostgreSQL/Redis 可用; 单元/集成测试; Git 与 Docker。
Agent	结构化输出; 3-5 个工具; 上下文预算; Memory 更新/过期/删除; 异常与无限循环保护。
Runtime	任务状态机; Worker 异步执行; Checkpoint; 暂停/取消/恢复; 外部写操作幂等; unknown outcome。
生产治理	超时、重试、退避、限流、熔断、降级; 权限、沙箱、密钥、审计、人工确认。
可观测性	端到端 Trace; 成功率、p95、队列、重试、Token、成本、模型路由和 Memory 误召回指标。
评测	固定测试集; 结果/轨迹/可靠性/安全/成本五层校验; 版本回归; badcase 可复现。
模型实验	Base vs Prompt/Schema vs LoRA SFT vs Distilled; 训练可恢复; 数据卡、模型卡和误差分析齐全。
Serving	vLLM 网关; TTFT/TPOT/吞吐/并发/Prefix Cache 实验; 限流、熔断、Fallback。
部署	docker-compose 一键启动; Helm/K8s 部署; Probe; KEDA/HPA; 故障注入与容量报告。
展示	架构图、状态流转、恢复视频、Dashboard、Eval 报告、成本优化、3-5 分钟演示和清晰简历表达。

最终项目必须回答的 6 个问题

- 模型为什么选择这个工具，低置信度或非法输出时如何回退？
- Worker、数据库、工具或 vLLM 故障后，系统如何恢复且不重复副作用？
- Memory 与 Context 如何处理作用域、冲突、过期、删除和 Token 预算？
- 训练是否真的优于 Prompt/Schema 优化，提升在固定测试集上是多少？
- 延迟、吞吐、Token 和单任务成本如何测量，什么时候用小模型、强模型或缓存？
- 为什么这里不需要复杂 Multi-Agent，何时单 Agent 才真的不可维护？

停止条件：当一个新技术不能在两天内变成可运行能力、自动测试、故障证据或面试故事时，先不学。最终目标不是覆盖所有名词，而是用一个系统证明：安全、可靠、可恢复、可观测、可评测、可部署。

PART V

模型开发、训练与项目闭环

把模型原理、数据工程、后训练、检索模型、评价模型、推理服务和 Durable Agent Runtime 串成一个可训练、可评测、可部署、可持续迭代的作品集。

阶段	核心产物	对应项目
理解模型	Tiny Transformer、生成与训练系统实验	Project A
构造数据	Data Engine、Dataset Card、污染检查、CPT	Project B
后训练	SFT、LoRA/QLoRA、蒸馏、偏好优化	Project C
检索与评价	Embedding、Reranker、Verifier、Reward Model	Project D/E
服务与闭环	量化、vLLM、路由、MLOps、Agent 数据回流	Project F/G

最终简历不堆七个孤立仓库。对外包装为三个项目：Agent Model Factory、Multi-Model Inference Platform、Secure Durable Agent Platform；仓库内部保留完整实验和可复现证据。

32. 本次增补范围与硬件现实边界

当前机器: Alienware m18 R1, RTX 4090 Laptop GPU (16GB 显存)、i9、64GB 内存、约 1TB WSL 可用磁盘。WSL2 已能识别 GPU; Ubuntu 侧尚未安装 PyTorch、pip 与 nvcc。训练、vLLM 和 AI Infra 主环境统一放在 Ubuntu/WSL2。

本地能力边界

模型规模	推荐方式	适合作为
20M - 100M	从零预训练; 完整实现训练循环	Tiny Transformer 原理与训练系统实验
0.5B - 0.6B	小规模全参数 / CPT / LoRA	继续预训练、训练恢复、消融实验
1.5B - 1.7B	QLoRA SFT、DPO、输出蒸馏	Tool Router 主训练模型
3B - 4B	QLoRA; 控制序列长度和 batch	增强版工具调用与部署实验
7B - 8B	4-bit 推理或极限 QLoRA	挑战项, 不作为最短路径阻塞项
14B+ / 多机多卡	本地不做	仅理解或后期短租云 GPU

环境原则

- Windows PyTorch 保留, 但模型训练和服务使用 Linux 独立环境, 互不冲突。
- 项目代码放在 ~/projects, 不要放在 /mnt/c 作为训练主目录。
- Training、Serving、Runtime 分开虚拟环境, 避免 PyTorch、vLLM、量化库和 FlashAttention 版本互相锁死。
- 没有编译自定义 CUDA 算子前, 不要求 nvcc; 不要在 WSL 内安装 Linux NVIDIA 驱动。
- 优先形成可复现小实验和量化指标, 再放大模型或租多卡。

最快成果顺序

环境自检 → Tiny Transformer 最小实验 → Tool-Use QLoRA SFT
→ 输出蒸馏 / DPO → Embedding / Reranker → Verifier
→ vLLM 网关 → Durable Agent 接入 → Badcase 回流

33. 完整知识地图（上）：模型、数据与训练工程

模块	必须掌握	可验证产出
模型结构	Tokenizer、Embedding、Self-Attention、Q/K/V、Causal Mask、RoPE、RMSNorm/LayerNorm、MLP、Residual、LM Head、Logits	Project A 中手写最小 Decoder-only Transformer
生成原理	自回归生成、Greedy、Temperature、Top-k、Top-p、停止条件、重复惩罚、KV Cache	生成函数与延迟/质量对照实验
数据工程	来源与许可证、清洗、去重、质量评分、PII、切分、污染/泄漏、版本、Dataset Card	Project B Data Engine
预训练/CPT	Causal LM、Packing、Batch/Token、Optimizer、Warmup、Weight Decay、梯度裁剪、灾难性遗忘	0.6B 领域继续预训练与 Base/SFT/CPT 对照
训练工程	FP32/FP16/BF16、梯度累积、Gradient Checkpointing、OOM、Checkpoint、优化器/随机状态恢复、可复现	训练中断恢复和显存/吞吐基准
分布式训练	DDP、FSDP、ZeRO、数据/张量/流水线并行、NCCL、分布式 Checkpoint 的职责边界	本地讲原理；后期双卡最小实验

训练项目的最低证据标准

- 保存训练配置、Git commit、数据版本、随机种子和环境锁文件。
- 同时记录训练 loss、验证指标、显存峰值、tokens/s、样例输出和失败日志。
- 至少有一个恢复实验：杀死进程后恢复模型、优化器、Scheduler、步数与随机状态。
- 所有模型结论必须来自 Base 与训练后模型的同一评测集对照，而不是只看 loss 下降。
- 数据集与评测集隔离，提供污染检查和删除/重建流程。

面试重点： 能解释张量如何流过模型、训练状态由哪些部分组成、为什么恢复不只加载权重，以及模型效果变化到底来自数据、目标函数还是推理配置。

34. 完整知识地图（下）：后训练、评测、服务与 Agent Infra

模块	知识范围	落地位置
SFT / PEFT	Chat Template、Loss Mask、多轮/Tool Calling 数据、LoRA/QLoRA、Rank、Alpha、Target Modules、Adapter 合并	Project C
蒸馏	输出/序列蒸馏、Logits 蒸馏、特征蒸馏、Teacher/Student、温度、数据筛选、Tokenizer 差异	Project C
偏好与 RL	Chosen/Rejected、DPO、ORPO、KTO、Reward Model、RLHF/PPO 概念、GRPO、可验证奖励、Reward Hacking	Project C/E
检索模型	Embedding、对比学习、In-batch/Hard Negative、Cross-Encoder Reranker、混合检索、Reranker 蒸馏	Project D
评价模型	Verifier、Outcome/Process Reward、分类、Pairwise Ranking、校准、Best-of-N	Project E
模型评测	Accuracy/F1、Recall@K、MRR、nDCG、Pass@K、鲁棒性、安全、校准、消融、成本/延迟	统一 Eval Hub
推理优化	BF16/FP16、bitsandbytes、GPTQ、AWQ、KV/Prefix Cache、Continuous Batching、TTFT、TPOT、Admission Control	Project F
MLOps	实验追踪、Dataset/Model Registry、模型卡/数据卡、Eval 门禁、灰度、回滚、漂移与 Badcase	Project F
Agent/AI Infra	Tool Use、MCP、Context、Memory/RAG、Durable Execution、幂等、审批、沙箱、OTel、K8s/KEDA	Project G
安全治理	Prompt Injection、工具最小权限、拒绝数据、安全微调、审批审计、红队评测	Project C/E/G

覆盖不等于每项都做成大型研究。SFT、QLoRA、输出蒸馏、DPO、Embedding、Reranker、Verifier、量化、vLLM、Eval 和 Agent 接入要有真实结果；多机多卡、完整 PPO、特征蒸馏、CUDA/Triton 做最小实验或清晰 Trade-off 即可。

35. 总项目：Agent Model Factory × Durable Runtime

一条贯通全部项目的数据闭环



仓库建议

```
agent-model-factory/
├── labs/tiny-transformer/      # Project A
├── data-engine/               # Project B
├── post-training/tool-router/ # Project C
├── retrieval/                 # Project D
├── verifier/                  # Project E
├── serving/                   # Project F
├── agent-runtime/             # Project G
├── evals/                     # 统一评测与回归
├── infra/                     # Docker / K8s / OTel
└── docs/                     # ADR / Model Card / 实验报告
```

对外简历包装

简历项目	内部覆盖	核心卖点
Agent Model Factory	A+B+C+D+E	数据、训练、蒸馏、偏好、检索模型、Verifier 与统一评测
Multi-Model Inference Platform	F	量化、vLLM、缓存、路由、限流、性能和 MLOps
Secure Durable Agent Platform	G + 全部模型接入	MCP、恢复、幂等、审批、Memory、Trace、K8s 和数据回流

主项目不是“调用多个模型的 Demo”，而是证明：模型如何被训练、如何被可靠部署、如何参与真实工具任务，以及线上失败如何变成下一版数据。

36. Project A: Tiny Transformer & Training Systems Lab

定位：用一个小模型覆盖模型结构、生成原理和训练状态，不追求榜单效果，追求能从源码解释与恢复。

必须实现

- Tokenizer（可先使用现成 BPE，再补最小 tokenizer 实验）。
- Embedding、Multi-Head Causal Attention、RoPE、RMSNorm、MLP、Residual、LM Head。
- 训练 forward、Cross Entropy、loss mask；生成 greedy / temperature / top-k / top-p。
- KV Cache 推理，并对比缓存前后的生成延迟。
- BF16/FP16、梯度累积、梯度裁剪、Warmup、学习率调度、Gradient Checkpointing。
- Checkpoint 保存模型、优化器、Scheduler、Scaler、步数和随机状态。

实验与验收

实验	指标/证据	验收
结构正确性	shape trace、单元测试、causal mask 可视化	无未来 token 泄漏
生成策略	样例、重复率、延迟	至少三种策略可复现
训练恢复	中断前后 loss/step/seed	恢复后连续训练
显存实验	峰值显存、tokens/s	解释 accumulation/checkpointing 权衡
KV Cache	TFT/逐 token 延迟	展示缓存收益与显存代价

简历/面试价值

作为 Agent Model Factory 的基础实验：证明理解 Decoder-only Transformer 的数据流、训练状态和推理缓存，而不是仅会调用 Trainer。

规模建议：20M – 80M 参数，先在小语料上数小时内完成训练；该项目不占简历第一项目位置，但仓库、测试与实验报告必须完整。

37. Project B: Domain Data Engine & Continued Pretraining

定位： 建立“数据可以被审计、重建、删除和版本化”的训练入口，并用 0.5B - 0.6B 模型完成小规模领域继续预训练。

领域优先级

领域	数据来源	可评测任务
Agent / MCP / AI Infra	官方文档、项目 ADR、Trace、故障复盘	术语问答、工具选择、故障分类
软件工程	代码、Issue、PR、测试日志	Issue→文件、补丁规划、错误解释
DevOps	日志、Runbook、告警、复盘	告警→Runbook、故障原因、操作风险

数据流水线

Source Manifest → License Check → Parse → Normalize → Deduplicate
→ Quality / PII → Chunk / Pack → Split → Contamination Check
→ Dataset Version → Dataset Card / Quality Report

必做对照

模型	目的	评测
Base	原始基线	领域/通用任务
CPT	领域语料继续预训练	领域 perplexity 与任务变化
SFT	仅指令微调	指令遵循和任务指标
CPT + SFT	两阶段组合	领域收益与通用能力保留

关键风险

- 训练/评测污染与近重复；数据新鲜度和来源许可证。
- 灾难性遗忘：领域指标上升但通用能力明显下降。
- 只追求语料量，缺少质量评分和可删除性。
- 把 CPT 与“让模型记住几篇文档”混为一谈。

38. Project C: Agent Tool-Use Post-Training & Distillation

核心模型项目：训练 1.5B - 1.7B 小模型完成工具选择、参数生成、风险判断、审批/拒绝和强模型回退。该项目直接回答“是否有微调、蒸馏经验”。

数据组成

数据源	作用	质量门禁
人工种子	定义正确格式与边界	逐条审查、工具可执行
教师模型合成	扩大覆盖和难度	Schema、执行验证、去重、一致性
真实 Agent Trace	贴近线上分布	脱敏、成功/失败标注、版本记录
扰动/安全样本	构造 rejected 与 hard cases	错误工具、危险参数、绕过审批、循环

训练矩阵

阶段	必须做	补充实验
Baseline	Base 模型统一评测	Prompt-only / few-shot
SFT	LoRA 与 QLoRA 对照	rank/alpha/target modules/data size 消融
蒸馏	教师输出/序列蒸馏	可访问 logits 时做 KL + hard loss 最小实验
偏好优化	DPO	ORPO 或 KTO 选一; GRPO 做可验证奖励小实验
接入	替换 Agent Tool Router	低置信度 fallback、审批与拒绝策略

统一指标

- Tool Accuracy、JSON Schema 合法率、参数字段 F1 / Exact Match。
- 风险分类 Macro F1、审批判断准确率、危险请求拒绝率与误拒率。
- Agent 最终任务成功率、平均步骤数、强模型回退率、延迟和成本。
- Base / SFT / Distilled / DPO 在同一测试集上的对照与 Badcase 分类。

不要把“教师 API 生成数据”说成 logits 蒸馏。API 通常只能做输出/序列蒸馏；logits 蒸馏需要访问教师分布，并处理温度、loss 权重和 tokenizer 对齐。

39. Project D: Embedding & Reranker Training

定位： 从“会接向量库”升级到能构造训练样本、挖困难负样本、微调检索模型并量化其对 RAG/Memory 的真实影响。

推荐任务

Query	Positive	Hard Negative
Issue / 用户问题	相关代码文件或技术文档	主题相似但不能解决问题的文件
告警	对应 Runbook	同类服务的其他故障手册
Agent 当前任务	有效项目 Memory	关键词相似但已过期/冲突的 Memory

训练与对照

BM25
vs Base Embedding
vs Fine-tuned Bi-Encoder (In-batch / Hard Negative)
vs Hybrid Search
vs Hybrid + Cross-Encoder Reranker

指标与落地

层面	指标	最终证明
召回	Recall@5/10、Top-1	相关内容能否进入候选集
排序	MRR、nDCG@10	正确内容是否排在前面
系统	延迟、索引大小、rerank 成本	在线可用性
Agent	任务成功率、过期 Memory 误召回率	检索改进是否改变最终任务

进阶但不阻塞

- Reranker 蒸馏到更轻的模型。
- 从线上 Badcase 自动挖 Hard Negative。
- Memory 版本、valid_to、superseded_by 与检索过滤联合评测。

40. Project E: Agent Verifier & Reward Model

定位：训练模型判断一条 Agent 轨迹是否正确、安全、值得继续；用于 Best-of-N、提前终止、重试/回滚和偏好数据构造。

输入与输出

输入：用户任务 + 计划 + Tool 调用序列 + Tool 结果 + 最终答案
输出：success_probability / error_type / unsafe_action
should_retry / should_rollback / pairwise_preference

负样本生成

- 替换错误工具、修改参数、删除必要步骤、打乱顺序。
- 重复产生副作用、绕过审批、使用过期 Memory。
- 工具失败却声称成功、最终答案与证据矛盾。
- 加入“看起来合理但证据不足”的困难负样本，避免只学格式。

训练任务与指标

任务	模型形式	指标
成功/失败	分类器或小生成模型	Accuracy、Macro F1、AUROC
错误类型	多分类	Macro F1、混淆矩阵
轨迹优劣	Pairwise Ranking / Reward Model	Pairwise Accuracy
概率可信度	校准	ECE / reliability diagram
系统收益	Verifier 门禁	漏放率、误杀率、Agent 成功率变化

Reward Model 不是“让另一个 LLM 随便打分”。需要明确标注协议、困难负样本、校准和线上决策阈值，并分别评估过程监督与结果监督。

41. Project F: Multi-Model Serving & MLOps Platform

定位： 将前面的模型部署为统一服务，证明能处理并发、过载、量化、路由、版本和回滚，而不是只在 Notebook 中推理。

服务架构

```
Client → OpenAI-compatible Gateway
├── API Key / Tenant / Quota / Rate Limit
├── Timeout / Retry / Circuit Breaker
├── Model Router / Confidence / Fallback
├── Cache / Metrics / Trace
└── vLLM Model Pool
```

必须实验

实验	变量	记录
并发	1 / 4 / 8 / 16 / 32	吞吐、p50/p95、TIFT、TPOT、错误率
Prefix Cache	相同 vs 不同前缀	命中率、TIFT、GPU 利用率
输出长度	不同 max_tokens	总延迟、TPOT、吞吐
量化	BF16/FP16、bnb 4/8bit、GPTQ、AWQ	显存、速度、质量变化
路由	全强模型 vs 大小模型分流	成功率、延迟、强模型比例、成本
过载	队列上限 / Admission Control	拒绝、排队、OOM 和恢复

MLOps 交付物

- MLflow/W&B：实验记录；数据、模型与配置版本绑定。
- Dataset Card、Model Card、训练报告、评测报告与已知限制。
- Eval 门禁、灰度发布、自动回滚、模型版本可追溯。
- 线上 Trace、漂移/Badcase 监控与训练数据回流队列。

42. Project G: Secure Durable Agent Platform（最终整合）

最终主项目：把 Tool Router、Retriever、Reranker、Verifier 和 Serving 平台接入可恢复 Agent Runtime，展示模型开发与 AI Infra 是同一条生产链路。

运行链路



必须具备

能力	实现	演示证据
可靠执行	异步 Worker、状态机、Checkpoint、取消/超时	杀死 Worker 后恢复
副作用治理	幂等键、unknown outcome、重复消息去重	重复投递不重复提交
安全	风险分级、HITL、最小权限、沙箱、审计	高风险步骤暂停批准
Context/Memory	预算、压缩、作用域、冲突、过期、删除	旧 Memory 不进入活动上下文
可观测性	OTel、Prometheus、Grafana	模型/工具/恢复端到端 Trace
部署	Docker、K8s、Helm、KEDA	积压驱动 Worker 扩缩
训练闭环	Trace→Badcase→审核→数据版本→训练→Eval	新版本有可量化提升

最终三分钟演示

- 提交一个多步骤桌面/研发任务；小模型完成路由和参数生成。
- 中途杀死 Worker，新 Worker 从 Checkpoint 接管。
- 最终提交触发人工审批；重复投递不会产生重复副作用。
- 打开 Trace/Grafana，展示延迟、错误、Token、模型版本和恢复事件。
- 展示一个 Badcase 如何进入数据集并在新模型中通过回归。

43. 最快实施顺序：21 天简历版与 6 周完整版

21 天简历可用版（按高强度投入）

时间	重点	必须产出
Day 1-3	环境 + Tiny Transformer	CUDA/PyTorch 验证、训练恢复实验、结构测试
Day 4-6	Data Engine + CPT 最小实验	Dataset Card、质量报告、Base/CPT 对照
Day 7-11	Tool-Use QLoRA SFT	统一 Eval、LoRA/QLoRA 消融、模型 Adapter
Day 12-14	输出蒸馏 + DPO	教师数据门禁、Base/SFT/Distilled/DPO 对照
Day 15-17	Embedding + Reranker	Hard Negative、Recall/MRR/nDCG 报告
Day 18-19	Verifier	分类/排序/校准与 Agent 门禁实验
Day 20-21	vLLM + Agent 接入	网关、路由、性能基准、端到端 Demo

第 4-6 周补齐

周	补齐内容	验收
第 4 周	MLOps、Eval 门禁、数据回流、Model Registry	可灰度、可回滚、版本可追溯
第 5 周	Durable Runtime、Memory、安全、OTel	10 个故障实验
第 6 周	Docker/K8s/HeIm/KEDA、负载与演示	一键部署、扩缩、容量报告、视频

若每天投入少于 6 小时，不压缩验收标准，而是延长周期。优先顺序固定：Tool-Use 后训练 → Eval → 接入真实 Agent → Serving/Infra；GRPO、多卡和复杂 CPT 不阻塞首个简历版本。

44. 简历表达、量化指标与实施深度边界

简历项目 1: Agent Model Factory

构建面向 Agent Tool Use 的数据、后训练与评测流水线，结合人工样本、教师合成数据和真实 Trace 完成 QLoRA SFT、输出蒸馏及 DPO；同时训练领域 Embedding/Reranker 与轨迹 Verifier，并通过统一 Eval 比较 Base、SFT、Distilled 和偏好优化模型。

简历项目 2: Multi-Model Inference Platform

基于 vLLM 实现 OpenAI-compatible 多模型网关，支持租户限流、Token 配额、缓存、熔断、置信度路由与强模型 fallback；完成量化、并发、TTFT/TPOT、Prefix Cache 和成本-质量对照，并建立模型灰度与回滚流程。

简历项目 3: Secure Durable Agent Platform

设计可恢复 Agent Runtime，支持 Worker 故障接管、Checkpoint、工具幂等、unknown outcome、人工审批、Memory 版本治理和端到端 Trace；将训练后 Tool Router、Retriever 和 Verifier 接入真实 MCP 工具任务，形成 Trace→Badcase→训练→回归闭环。

必须填入真实数字

模型/系统	推荐数字
Tool Router	Tool Accuracy、JSON 合法率、风险 F1、回退率、Agent 成功率
Retriever	Recall@10、MRR、nDCG、过期 Memory 误召回率
Verifier	Macro F1、AUROC、ECE、错误轨迹漏放率
Serving	峰值显存、TTFT、TPOT、tokens/s、p95、吞吐、强模型比例
Runtime	恢复成功率、恢复时间、重复副作用数、审批命中率、故障覆盖数

深度边界

级别	内容
真实完成	Tiny Transformer、Data Engine、CPT 最小实验、SFT/QLoRA、输出蒸馏、DPO、Embedding、Reranker、Verifier、量化、vLLM、Eval、Agent 接入
最小实验	DDP/FSDP/ZeRO、logits 蒸馏、Reward Model、GRPO、模型灰度/回滚
理解与 Trade-off	多节点、NCCL/RDMA、Megatron、完整 PPO/RLHF、特征蒸馏、CUDA/Triton、张量/流水线并行

任何没有真实运行证据的能力，不写成“熟练掌握”或“生产经验”。简历强调问题、约束、方案、验证和权衡；框架名称只作为实现细节。